Phase 5: Architecture and Design

Course Code: 20CYS401

5.1 System Architecture Overview

The platform uses a **four-layer architecture** implemented in Python/Flask:

```
Browser Client
■
▼
■■■
```

```
end

subgraph "Security Layer"
S1[Flask-Login Session]
S2[CSRF — Flask-WTF]
S3[admin_required decorator]
S4[Security Headers Middleware]
S5[Audit Logger]
end

subgraph "Service Layer"
SV1[UserService]
SV2[BookService]
SV3[SearchService]
SV4[ReviewService]
SV5[ReadingListService]
end

subgraph "Data Layer — SQLAlchemy ORM"
M1[User Model]
M2[Book Model]
M3[Review Model]
M4[ReadingListItem Model]
M5[AuditLog Model]
end
end

subgraph "Database"
DB[(SQLite book_platform.db)]
end

Browser -->|HTTPS Request| R1
Browser -->|HTTPS Request| R2
Browser -->|HTTPS Request| R3
Browser -->|HTTPS Request| R4
Browser -->|HTTPS Request| R5

R1 --> S1
R3 --> S3
R5 --> S3
R4 --> S1

R1 --> SV1
R2 --> SV2
R2 --> SV3
R3 --> SV4
R4 --> SV5
R5 --> SV2
R5 --> SV4

SV1 --> M1
SV2 --> M2
SV3 --> M2
SV4 --> M3
SV4 --> S5
SV5 --> M4
SV2 --> S5

M1 --> DB
M2 --> DB
M3 --> DB
M4 --> DB
M5 --> DB

S5 --> M5
```

5.3 Component Inventory

| ID | Component | File | Responsibility |

|---|---|---|---|

| COMP-01 | Application Factory | `app/__init__.py` | Creates Flask app, registers blueprints, sets security headers |

| COMP-02 | Config | `app/config.py` | Environment-based configuration with absolute DB path |

| COMP-03 | Data Models | `app/models/` | ORM entities: User, Book, Review, ReadingListItem, AuditLog |

| COMP-04 | Services | `app/services/` | Business logic, ownership checks, IDOR prevention |

| COMP-05 | Routes | `app/routes/` | HTTP request/response handling, input intake |

| COMP-06 | Security Core | `app/core/security.py` | RBAC decorator, input validators |

| COMP-07 | Audit Core | `app/core/audit.py` | Tamper-evident event logging with secret scrubbing |

| COMP-08 | Templates | `app/templates/` | Jinja2 server-rendered UI with autoescaping |

5.4 Security Architecture Decisions

| Decision | Rationale | Alternatives Rejected |

|---|---|---|

| **Server-side sessions** via Flask's signed cookie | Prevents client-side session tampering; session contents are not readable by client | JWT — introduces signature algorithm confusion attacks, token revocation complexity |

| **Bcrypt rounds=12** | Computationally expensive to brute-force | MD5, SHA-256 — fast hash functions not suitable for password storage |

| **Object-level authorization in Service layer** (not route layer) | Authorization logic testable independently; cannot be bypassed by changing HTTP method | Route-layer check only — easily bypassed by alternate endpoints |

| **SQLAlchemy ORM** for all queries | Parameterized by design; eliminates SQL injection class | Raw `db.execute(f"...")` — vulnerable to SQLi |

| **CSRF via Flask-WTF** hidden form tokens | Prevents cross-site form submission from malicious origins | None / `SameSite=Lax` only — insufficient for some browser versions |

5.5 Deployment Architecture

```
graph LR
subgraph "Local Development"
DEV[Flask Dev Server\nwsgi.py :5000]
DEVDB[(SQLite\nbook_platform.db)]
end

subgraph "Container"
DOCK[Docker Container\nnon-root user\nflask run\n:8080]
DOCKDB[(Volume Mount\ndb directory)]
end

subgraph "Kubernetes"
NS[Namespace: bookplatform]
DEP[Deployment\nreplicas: 2]
SVC[Service: ClusterIP]
CM[ConfigMap]
SEC[Secret: flask-secret]
HP[Health\n/health liveness probe]
end

DEV --> DEVDB
DOCK --> DOCKDB
NS --> DEP
NS --> SVC
DEP --> CM
DEP --> SEC
```

```
DEP --> HP
```